

广州城建职业学院

GUANGZHOU CITY CONSTRUCTION COLLEGE

# 毕业实践综合项目设计

# 说 明 书

| | |
|---|---|
| 作品名称： | 基于 Vue+Node.js 的超市商品管理系统设计与实现 |
| 学生姓名： | 吴东盛 |
| 学生学号： | 2435102010339 |
| 所在学院： | 信息与智能工程学院 |
| 班　级： | 24 计应 1 班 |
| 指导教师： | 李杰 |

# 毕业实践综合项目原创性声明

本人郑重声明：所呈交的毕业实践综合项目报告书是本人在指导教师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本毕业实践综合项目报告书不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：吴东盛　　日期：2026 年 11 月 5 日

# 作品信息概要表（2023 版）

| | | | |
|---|---|---|---|
| **学生姓名** | 吴东盛 | **作品名称** | 基于 Vue+Node.js 的超市商品管理系统设计与实现 |
| **作品方向** | 编程 | **作品类型** | 网站 / Web 应用 |
| 作品简介（100 字以内）：面向单店超市的进销存后台管理系统，覆盖商品、供应商、库存、采购、销售、退货、盘点和经营分析，支持六类岗位权限与库存事务一致性。 | | | |
| 创新描述（100 字以内）：以单据驱动库存并全程留痕；事务、行锁与固定锁序保证一致性；成本快照与加权平均成本结合；权限下沉到接口并做字段级脱敏；单元测试、冒烟测试与浏览器 E2E 可验证。 | | | |
| **制作平台** | ☑Windows ☐Linux ☐macOS ☐其他： | **展示平台** | ☑Windows ☐Linux ☐macOS ☐iOS ☐Android |
| **制作工具** | VS Code、Navicat、Git、Chrome DevTools、WPS Office | | |
| **参考文献、项目或作品（前 3 项）** | [1] 袁明坤,曾丽.基于 JavaWeb 的超市商品采购管理系统设计与实现[J].电脑知识与技术,2022,18(5):61-63.<br>[2] 杨正午.基于 WEB 前端开发技术的网站设计——以连锁超市商品销售管理系统为例[J].山西科技,2019,34(3):51-53+57.<br>[3] 赵霁雯.中小型超市库存管理系统设计及实现[D].大连:大连理工大学,2018. | | |
| **提交内容** | ☐素材压缩包 ☑报告文档 ☑演示视频 ☐PPT ☑源代码 ☑部署文件<br>□数据集 □模型 □作品文件 □其他 | | |

填写说明：所有□可根据需要变化为■（软键盘输入）。

# 基于 Vue+Node.js 的
# 超市商品管理系统设计与实现

## 摘 要

针对中小型超市仍依赖手工登记或 Excel 管理商品、库存与销售，存在录入效率低、账实不符、库存预警滞后和经营数据难以统计等问题，本文设计并实现了一套基于 Vue + Node.js 的超市商品后台管理系统。系统采用前后端分离架构，前端使用 Vue 3、Vite、Element Plus、Pinia 与 ECharts，后端使用 Node.js、Express 5 与 MySQL 8，按照路由层、控制层、服务层、数据访问层分层组织。系统实现商品与分类、供应商、库存出入库、采购、收银销售、整单退货、库存盘点和经营分析等功能，设计 14 张业务表和 69 个接口，并建立管理员、仓管、收银员、采购、后勤、会计六类岗位的权限矩阵。针对库存一致性，系统采用单据驱动库存、MySQL 事务、按商品 id 升序加行锁和移动加权平均成本；针对数据安全，采用 JWT、bcrypt、token 版本号和财务字段脱敏。后端 105 个单元测试全部通过，12 步浏览器端到端流程验证通过，并接入 GitHub Actions 持续集成。结果表明，系统能够完成从商品建档到采购、销售、退货、盘点的完整业务闭环。

**【关键词】**Vue 3；Node.js；Express；MySQL；进销存管理；权限控制

# 目 录

1.需求分析.....1
1.1 项目背景与问题.....1
1.2 同类系统对比.....1
1.3 目标用户与功能需求.....2
1.4 非功能需求.....2
2.概要设计.....3
2.1 总体架构.....3
2.2 功能模块划分.....4
2.3 技术选型.....4
2.4 接口与数据规范.....5
3.详细设计.....6
3.1 数据库设计.....6
3.2 关键业务设计.....7
3.3 权限与安全设计.....11
3.4 前端设计.....12
4.展示和测试.....13
4.1 运行环境与启动方式.....13
4.2 主要界面展示.....14
4.3 测试方案与结果.....17
4.4 测试中发现并解决的问题.....19
5.创新点与不足.....21
5.1 主要特点.....21
5.2 不足与改进方向.....21
6.项目总结.....23
7.参考文献.....24
8.致谢.....25

# 1.需求分析

## 1.1 项目背景与问题

中小型超市的商品管理涉及商品档案、采购入库、销售收银、退货、盘点和供应商结算等多个环节，数据量大且变化频繁。观察本校周边小型超市可见，相当一部分门店仍采用纸质台账或 Excel 表格记录商品和库存：商品名称、规格、进价和售价分散在多个文件中；每日销售结束后由店员手工汇总；库存靠“上次数值加进货减销售”估算；盘点时再集中核对。这种管理方式在商品数量较少时还能维持，但随着品类增加，问题逐渐显现[1-2]。

一是录入效率低，同一商品在采购、入库和销售单中重复录入；二是账实不符，销售与库存分别记录，退货、赠品和破损没有统一入口；三是预警滞后，缺货和积压只能靠人工巡店发现；四是经营数据难统计，销售额、毛利和分类占比靠手工计算，成本口径不统一；五是操作缺少留痕，改价、退货和盘点审核的责任边界不清晰。

本课题面向单店超市内部管理，设计基于 Vue + Node.js 的商品后台管理系统，把商品、供应商、库存、采购、销售、退货和盘点纳入同一数据模型，做到每笔库存变动有单据和流水、每个关键动作有操作日志，为经营者提供可核对的销售与库存数据。

## 1.2 同类系统对比

同类产品大致分为两类。一类是面向连锁企业的通用进销存或 ERP 系统，功能覆盖采购、销售、库存、财务和门店管理，但模块多、配置复杂，需要专门培训，部署和维护成本对单店而言偏高。另一类是电商平台提供的商家后台，优势是与线上订单打通，但业务重心是电商订单和物流，对线下收银、现金找零、整单退货等场景支持有限。课程作业类系统则多集中在商品增删改查，缺少库存流水、事务一致性和权限隔离[3-5]。

表 1.1 从目标场景、部署成本、业务闭环、权限与安全和二次开发五个维度比较三类方案。本系统不追求“大而全”，而是把单店高频业务做完整：商品建档、期初入库、收银开单、退货、盘点、采购收货与付款形成闭环，同时保留清晰的接口和分层结构。

表 1.1 同类系统对比

| 维度 | 通用连锁进销存 / ERP | 电商商家后台 | 本系统 |
|---|---|---|---|
| 目标场景 | 多门店、连锁经营 | 线上订单为主 | 单店线下经营 |
| 部署与使用成本 | 高，需实施与培训 | 低，但依赖平台 | 低，Node + MySQL 即可部署 |
| 业务闭环 | 全面但配置复杂 | 订单、发货、退款 | 商品—采购—销售—退货—盘点闭环 |
| 权限与数据安全 | 权限细，但成本高 | 由平台控制 | 六类岗位 + 字段级脱敏 |
| 二次开发 | 封闭或需授权 | 受限 | 前后端分离，源码可扩展 |

## 1.3 目标用户与功能需求

系统面向六类岗位：管理员负责全局配置、用户管理、采购付款、退货确认和盘点审核；仓管负责出入库、采购收货和盘点制单；收银员负责开单和发起退货；采购负责采购制单；后勤只读查询商品、库存和看板；会计查看全部销售单、库存与盘点。不同岗位的菜单、接口和财务字段可见范围不同。

功能划分为八个模块：运营总览、商品管理、库存管理、采购管理、销售管理、供应商管理、系统管理、权限与安全。核心业务闭环有三条：一是“商品建档 → 期初入库 → 销售开单 → 退货申请/确认 → 回补库存”；二是“供应商 → 采购制单 → 提交 → 仓管收货入库 → 生成应付 → 登记付款”；三是“盘点任务 → 实盘录入 → 提交 → 管理员审核 → 盘盈/盘亏修正库存”。

## 1.4 非功能需求

（1）数据一致性。销售、入库、退货、作废和盘点审核会同时修改单据、库存和流水，三者必须在同一事务内完成，失败整体回滚；并发结账不能超卖。

（2）安全性。密码 bcrypt 加密；接口 JWT 鉴权；改密、重置、改岗位或禁用后旧 token 立即失效；成本、毛利、库存总值、采购金额按岗位脱敏，不能只靠前端隐藏。

（3）性能与可用性。列表接口支持分页和筛选；条码、销售单号、采购单号建立唯一索引；课程设计规模下保持毫秒级响应；页面在 1366×768 及以上正常使用。

（4）可维护性。后端保持路由、控制、服务、数据访问分层；接口返回统一结构；关键业务提供单元测试、冒烟测试和浏览器 E2E，并接入持续集成。

# 2.概要设计

## 2.1 总体架构

系统采用前后端分离架构，如图 2.1 所示。浏览器加载 Vue 3 单页应用，页面通过 Axios 调用后端接口；Express 5 接收请求后依次经过 CORS、JSON 解析、JWT 鉴权、岗位权限校验和登录限流等中间件，再进入路由层。路由只负责 URL 分发，Controller 负责参数校验、业务编排和事务边界，Service 承载入库、退货等跨模块领域逻辑，Model 只负责 SQL 与数据持久化，最终读写 MySQL。所有接口返回统一响应体 { code, msg, data }，业务失败通过 BizError 抛出，由全局错误中间件转换为可展示的提示文案。

在部署上，开发阶段前端 5173、后端 3000 分别启动，Vite 将 /api 代理到后端；演示阶段可先构建前端，再由后端通过 SERVE_FRONTEND=1 托管 dist，实现单端口访问。图 2.1 中的“安全与公共能力”不属于某一层，而是贯穿接口层和业务层的横切能力，包括 JWT、岗位权限、登录限流、统一响应、错误处理、操作日志和财务字段脱敏。




图 2.1 系统总体架构图

## 2.2 功能模块划分

系统功能模块如图 2.2 所示[6-8]。八个模块按“基础资料—库存—业务单据—分析与系统管理”组织：商品管理和供应商管理维护基础资料；库存管理负责出入库、流水、预警和盘点；采购管理负责采购单从草稿到入库、付款的状态流转；销售管理负责收银开单、销售记录、退货单和销售统计；运营总览汇总今日/本月销售额、利润、7 日趋势、分类库存和热销商品；系统管理负责用户、个人中心和操作日志；权限与安全模块提供六类岗位的权限矩阵、路由与接口双重鉴权、字段级脱敏和旧 token 失效能力。

模块之间通过单据和库存流水关联，而不是直接修改对方的页面状态。例如销售开单只写销售单、销售明细和出库流水，商品库存由后端在同一事务内扣减；盘点审核只写盘点单状态、库存修正和盘盈/盘亏流水。这样每个模块的职责边界清晰，出现问题时可以沿单据号追溯。




图 2.2 系统功能模块图

## 2.3 技术选型

技术选型如表 2.1 所示。前端选择 Vue 3 的组合式 API[11]，配合 Vite 8 获得较快的冷启动和热更新；Element Plus 提供表格、表单、弹窗等后台常用组件；Pinia 管理登录用户和权限；ECharts 绘制销售趋势、分类库存和热销排行。后端选择 Express 5[9]，原因是中间件模型简单、路由清晰，适合把鉴权、限流、错误处理拆成独立中间

件；数据库选择 MySQL 8.0，使用 InnoDB 事务和行锁保证库存一致性；密码使用 bcryptjs，令牌使用 jsonwebtoken；测试使用 Node.js 原生测试运行器和 Playwright Core，减少额外依赖。

表 2.1 技术选型

| 层次 | 技术 / 版本 | 用途 | 选择理由 |
| --- | --- | --- | --- |
| 前端框架 | Vue 3.5 + Vite 8 | 页面与构建 | 组合式 API 易拆分，构建速度快 |
| UI 与状态 | Element Plus 2.14 + Pinia 4 | 后台组件、登录态 | 组件成熟，权限状态集中管理 |
| 图表 | ECharts 6 | 看板与统计图 | 图表类型全，可按需引入 |
| 后端 | Node.js ≥ 20 + Express 5 | 接口服务 | 中间件机制适合鉴权与错误处理 |
| 数据库 | MySQL 8.0（InnoDB） | 业务数据持久化 | 支持事务、行锁与唯一索引 |
| 安全 | JWT + bcryptjs + 限流 | 鉴权、加密、限流 | 轻量，易与 Express 集成 |
| 测试 | Node test runner + Playwright Core | 单元、E2E | 无需额外测试框架，贴近真实浏览器 |

## 2.4 接口与数据规范

接口统一以 /api 为前缀，按资源划分：/api/user、/api/category、/api/product、/api/supplier、/api/stock、/api/stocktake、/api/purchase、/api/sale、/api/dashboard、/api/operation-log。所有接口返回 HTTP 200，业务结果由响应体中的 code 表达：200 表示成功，401 表示登录失效，403 表示无权限，429 表示登录过于频繁，其余为业务失败。分页接口统一返回 { list, total, pageNum, pageSize }，日期筛选统一使用 startTime/endTime，业务时区固定为 Asia/Shanghai，MySQL 会话时区设置为 +08:00，避免“今天”的统计跨时区偏移。

数据库字段使用下划线命名，前端使用驼峰命名，转换在接口层完成；金额使用 DECIMAL 存储，成本保留 4 位小数，售价和金额保留 2 位小数；单号采用“前缀 + 日期 + 序号”的格式，如 XS20261103-001、CG20261103-001、PD20261103-001、TH20261103-001，并通过唯一索引兜底。

# 3.详细设计

## 3.1 数据库设计

### 3.1.1 概念结构设计

根据需求，系统抽象出商品、分类、供应商、库存流水、销售单、销售明细、退货单、退货明细、采购单、采购明细、盘点单、盘点明细、用户和操作日志共 14 个实体。图 3.1 给出核心业务实体的 E-R 图：分类与商品、供应商与商品为一对多关系；商品与库存流水、销售明细、采购明细、盘点明细均为一对多关系；退货明细复制销售明细的商品与成本快照；销售单与销售明细、退货单与退货明细为一对多关系；销售单与退货单为一对多关系；采购单与采购明细、盘点单与盘点明细为一对多关系；用户与操作日志为一对多关系。为保留历史事实，销售明细、退货明细、采购明细和盘点明细中的商品名称、分类名称、单位、售价、成本等字段均保存快照，即使商品后续改名或改价，历史单据仍按当时数据展示。

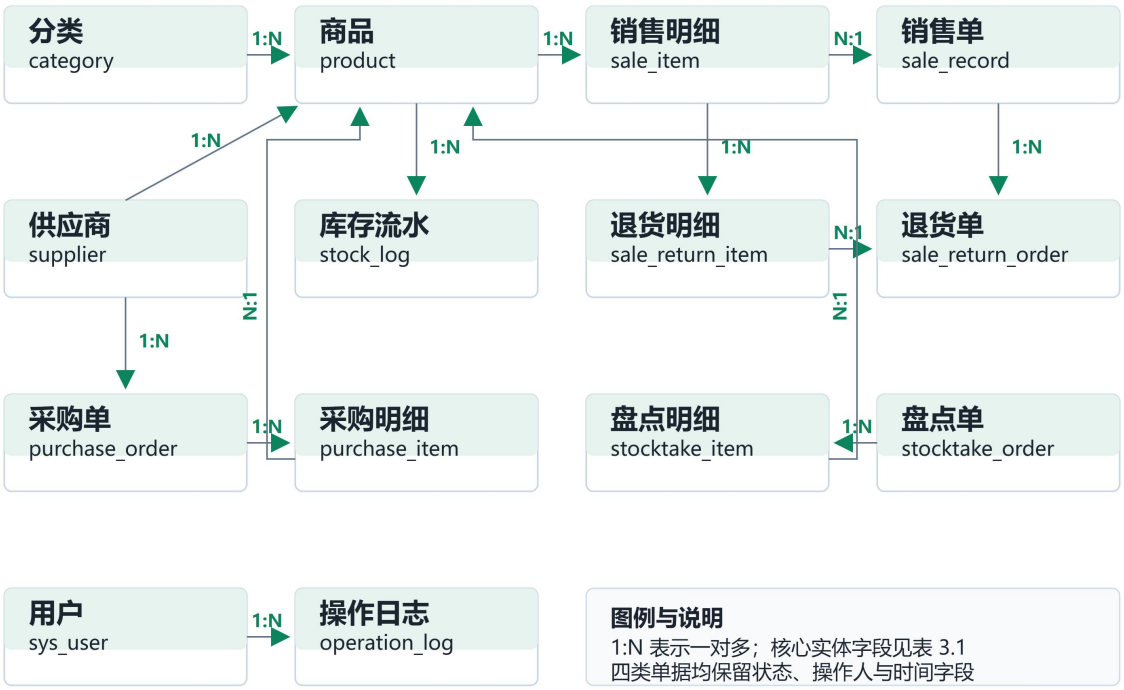



图 3.1 核心业务 E-R 图

### 3.1.2 逻辑结构设计

14 张表的逻辑结构如表 3.1 所示。所有表使用 InnoDB 引擎和 utf8mb4 字符集，主键统一为 BIGINT UNSIGNED 自增；金额字段使用 DECIMAL，库存和数量使用

INT；每张单据表保留 status、operator、create_time 等公共字段，便于状态流转和审计。唯一索引承担业务唯一性：用户名、商品条码、销售单号、退货单号、采购单号、盘点单号均唯一；采购明细、盘点明细使用 (order_id, product_id) 联合唯一，防止同一单据重复添加同一商品；销售单按 create_time 建索引，库存流水按 product_id 和 create_time 建索引，满足列表和统计查询。

表 3.1 数据表清单

| 序号 | 表名 | 作用 | 关键字段 / 约束 |
|---|---|---|---|
| 1 | sys_user | 系统用户 | username UK；position；token_version |
| 2 | category | 商品分类 | name UK；sort；status |
| 3 | supplier | 供应商 | supplier_name UK；contact_person |
| 4 | product | 商品档案 | product_code UK；cost_price；stock_num；warn_num |
| 5 | stock_log | 出入库流水 | product_id；change_type；biz_type；before/after_num |
| 6 | sale_record | 销售单 | sale_no UK；total_amount；pay_type；status |
| 7 | sale_item | 销售明细 | sale_record_id；product_id；cost_price 快照 |
| 8 | sale_return_order | 退货单 | return_no UK；sale_record_id；status |
| 9 | sale_return_item | 退货明细 | return_id；product_id；cost_price 快照 |
| 10 | stocktake_order | 盘点单 | take_no UK；status；盘盈/盘亏数量金额 |
| 11 | stocktake_item | 盘点明细 | order_id + product_id UK；book/actual/diff_qty |
| 12 | purchase_order | 采购单 | order_no UK；payable_amount；paid_amount |
| 13 | purchase_item | 采购明细 | order_id + product_id UK；quantity；price |
| 14 | operation_log | 操作日志 | module；action；biz_no；before/after_status；ip |

需要说明的是，当前建表脚本没有声明物理外键，表间关系由应用层在事务内校验（例如销售前校验商品存在、采购收货前校验商品存在），这样在重建演示库和导入种子数据时更灵活。代价是数据库层面无法完全阻止脏数据，这一点将在第 5 章作为不足说明。

## 3.2 关键业务设计

### 3.2.1 库存与成本模型

库存采用“单据驱动 + 流水留痕”的方式。任何库存变化都不直接改商品表，而是先产生业务单据，再在事务内更新 product.stock_num 并写入 stock_log。stock_log 记录 change_type（1 入库、2 出库）、biz_type、变动数量、变动前后库存、供应商、入库

单价、操作人和备注，因此可以还原任意商品在任意时点的库存变化。biz_type 共 8 类，如表 3.2 所示。

表 3.2 库存流水业务类型

| 业务类型 | biz_type | change_type | 说明 |
|---|---|---|---|
| 采购入库 | 1 | 1 | 采购收货时写入，同时更新加权成本 |
| 销售出库 | 2 | 2 | 收银结账时写入，按售价计营收、按成本快照计毛利 |
| 销售退货 | 3 | 1 | 退货确认时回补库存，不改变均价 |
| 盘点盘盈 | 4 | 1 | 审核盘点单时按实盘数大于账面数写入 |
| 盘点盘亏 | 5 | 2 | 审核盘点单时按实盘数小于账面数写入 |
| 期初入库 | 6 | 1 | 商品建档或期初录入时写入 |
| 手工调整 | 7 | 1 / 2 | 管理员手工修正库存 |
| 作废回补 | 8 | 1 | 销售作废时回补库存 |

成本采用移动加权平均法，公式为：新均价 =（入库前库存 × 原均价 + 入库数量 × 入库单价）÷（入库前库存 + 入库数量）。首次入库或入库前库存为 0 时，新均价直接取本次入库价；未提供入库价时维持原均价。销售、销售退货、盘盈盘亏都不改变均价：销售时把当时的成本写入 sale_item.cost_price 作为快照，毛利按“售价 − 成本快照”计算；退货时复制原销售明细的成本快照；盘点差异金额按审核时成本计算。这样同一批商品在不同时间销售，毛利不会被后续的采购价变化反向改写[12]。

为避免成本被误改，商品编辑接口不允许手工修改 cost_price：如果前端提交的成本与原值不一致，后端直接抛出“成本价由入库自动计算，不能手工修改”。商品因缺货下架后，一旦入库或退货回补使库存大于 0，系统会自动把商品重新置为在售。

### 3.2.2 销售开单与事务一致性

收银开单是系统中并发最高、数据一致性要求最强的场景。设计要点如下，流程如图 3.2 所示。

（1）合并明细。同一商品可能在购物清单中出现多行，后端先按商品 id 合并数量，再统一校验库存，避免逐行校验时“每行都不超库存、合计却超卖”。

（2）事务边界。整个结账过程放在一个 MySQL 事务中：校验商品、写销售单、写销售明细、扣库存、写库存流水、写操作日志，任意一步失败都回滚，不会出现“单据已生成但库存没扣”或“库存扣了但没有流水”的中间状态。

（3）行锁与锁顺序。校验库存前使用 SELECT ... FOR UPDATE 锁定商品行，并按商品 id 升序加锁。销售、退货回补、采购收货和盘点审核都遵循同一顺序，降低并发事务互相等待造成死锁的概率。

（4）价格与成本取数据库值。后端忽略前端传入的 salePrice，售价统一取 product.sale_price，成本取 product.cost_price 并写入明细快照，防止收银价被篡改。

（5）单号冲突重试。单号由“XS + 日期 + 序号”生成，并发下可能撞车；sale_no 上的唯一索引兜底，捕获重复键后做有限次重试，避免直接向用户抛数据库错误。

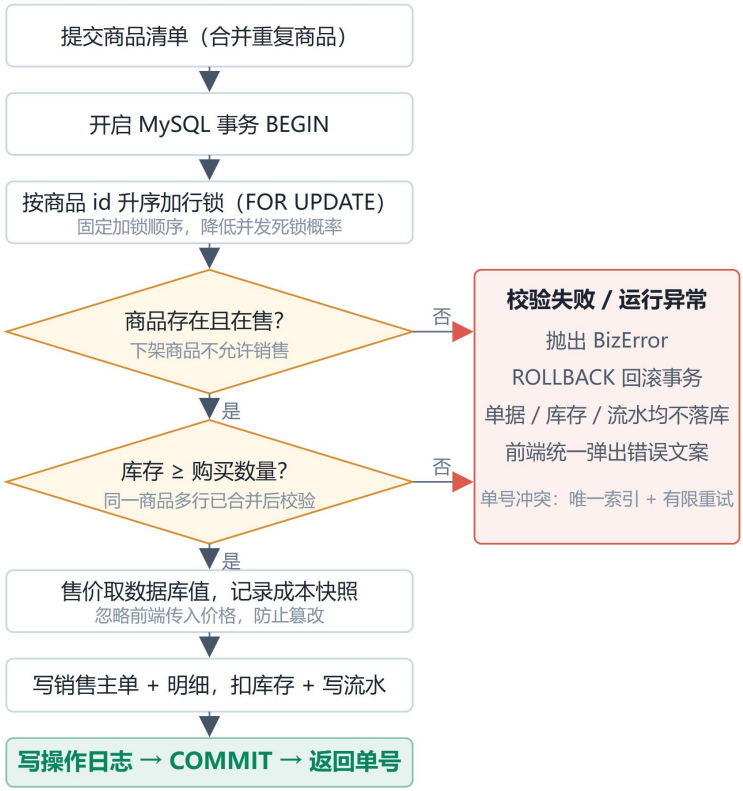



图 3.2 销售开单事务流程图

### 3.2.3 退货与作废

退货采用独立退货单，而不是直接修改原销售单。收银员在销售记录中发起整单退货，系统校验收银员只能为自己的销售单发起退货，且同一销售单不能存在进行中的退货单；申请时复制原销售明细作为退货明细快照，记录应退数量、原售价和原成本，此时不动库存、不改原单状态。管理员确认后，系统按“先锁原销售单、再锁退货单、最后按商品 id 升序锁商品”的顺序，回补库存、写销售退货流水、把原销售单置为已退货、把退货单置为已确认，并写操作日志。原销售单退出营收统计，等价于按原金额冲减营收；退货不改变加权平均成本。

退货单状态流转如图 3.3 所示：待确认可以转为已确认、已拒绝或已取消；已确认、已拒绝、已取消均为终态。销售作废时，系统会先锁原销售单，回补库存并写“作废回补”流水，同时把该单下待确认的退货申请自动取消，避免之后退货确认造成二次回补。

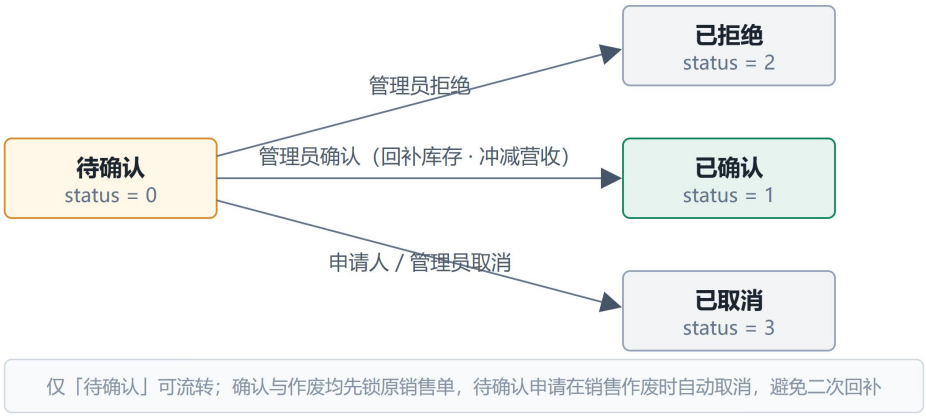



图 3.3 退货单状态流转图

### 3.2.4 采购与应付

采购单状态为 0 草稿、1 已提交、2 已入库、3 已取消。草稿可以编辑、提交和取消；提交后不可编辑；只有已提交的采购单可以由仓管或管理员收货入库；已入库和已取消的采购单不能再次取消。收货时系统按商品 id 升序锁定商品，调用与手工入库相同的 applyInbound 服务：增加库存、按采购单价更新加权平均成本、写采购入库流水；随后把采购单置为已入库，并把采购总金额写入应付金额。管理员登记付款时校验“只能对已入库单据付款、付款金额大于 0、不能超过未付金额”，付款后累计已付金额。

采购模块有意保持简单：应付金额挂在采购单上，只记录累计已付，不建立独立的付款流水表。对于单店课程设计场景，这样可以减少表数量和页面复杂度；代价是无法按付款时间、付款方式对账，这一限制在第 5 章说明。

### 3.2.5 盘点差异处理

盘点单状态为 0 草稿、1 已提交、2 已审核、3 已取消。创建盘点任务时按分类（可选全部）生成商品快照，记录账面数和默认实盘数；草稿阶段可以保存实盘数量和盈亏原因；提交前系统校验所有盈亏行必须填写原因；审核由管理员执行，审核时

重新读取商品当前库存和当前成本，按“实盘数 − 审核时库存”计算差异，避免从制单到审核期间发生销售或入库导致按旧账面数修正。

审核时逐商品加行锁，写盘点明细的审核时账面数、差异数量和差异金额，按差异方向修正库存并写盘盈或盘亏流水；盘盈使缺货下架商品恢复在售；盘盈盘亏不改变加权平均成本。审核完成后重算盘点单的盈亏总数量和总金额，并把状态置为已审核。

## 3.3 权限与安全设计

系统采用“角色 + 岗位”的双字段模型：role 取 admin/operator，用于后端判断是否管理员；position 取管理员、仓管、收银员、采购、后勤、会计，用于映射权限码。权限码按“资源:动作”命名，例如 product:read、stock:in、purchase:receive、sale:return:create、stocktake:audit、sale:read:all。管理员拥有全部权限；其他岗位的权限矩阵如表 3.3 所示。后端在路由上使用 requirePermission 校验权限码，前端在路由 meta 中声明所需权限，菜单和页面按钮再按权限码显隐，形成“菜单—路由—接口”三层控制。

表 3.3 岗位权限矩阵

| 岗位 | 商品 / 库存 | 采购 | 销售 | 盘点 | 用户 / 日志 |
|---|---|---|---|---|---|
| 管理员 | 全部 | 制单/收货/付款 | 全部 | 制单/审核 | 用户管理/日志 |
| 仓管 | 商品只读；入库/出库 | 查看/收货 | 无 | 查看/制单 | 无 |
| 收银员 | 商品只读 | 无 | 开单/本人单据/发起退货 | 无 | 无 |
| 采购 | 商品/库存只读 | 制单/提交/取消本人单据 | 无 | 无 | 无 |
| 后勤 | 商品/库存只读 | 无 | 无 | 无 | 无 |
| 会计 | 商品/库存只读 | 无 | 查看全部销售单 | 查看 | 无 |

安全方面还包括：密码使用 bcrypt（10 轮）加密；JWT 默认 24 小时过期，载荷中携带 tokenVersion；每次请求都回库校验账号状态和 tokenVersion，修改密码、重置密码、变更岗位或禁用账号后 tokenVersion 自增，旧 token 立即失效[13]；登录接口同一 IP 每分钟最多 5 次尝试，超限返回 code 429；CORS 只允许配置中的前端地址；生产

环境强制要求至少 32 位且非示例值的 JWT_SECRET；成本、毛利、库存总值、盘点盈亏金额、采购价格和应付金额按岗位做字段级脱敏，非管理员即使直接调用接口也拿不到敏感字段。

## 3.4 前端设计

前端采用“深墨青侧边栏 + 白色内容区 + 翡翠绿品牌色”的后台视觉方案，颜色、字号、圆角、阴影统一在 CSS 变量中定义，业务页面只引用变量。主布局由侧边菜单、顶部面包屑和内容区组成；菜单按权限动态显示，父级菜单负责分组，子路由使用 ParentView 承载。登录页采用左右分栏：左侧是深色品牌区，展示库存预警、出入库留痕、多角色协同等特性；右侧是登录表单，提供管理员、仓管、收银员三个演示账号的快捷填入，并在进入页面时探测后端健康状态。

路由守卫负责三件事：无 token 访问业务页时跳转登录并携带 redirect；已登录访问登录页时回到首页；每次整页加载通过 /api/user/info 刷新用户信息和权限，避免沿用本地缓存的旧权限。接口按模块放在 src/api/modules 下，Axios 响应拦截器统一解包 data，遇到 401 清除登录态并跳转登录页，其他业务错误直接弹出后端返回的 msg。

公共能力方面，分页参数和页码切换封装为 usePagination，ECharts 初始化与自适应封装为 useEchart，扫码枪输入封装为 useBarcodeScanner；PagePagination 和 StatusTag 统一分页条与状态标签；商品、库存和销售页面支持 CSV 导出。收银页支持按名称/条码搜索、分类筛选、数量调整、实收与找零计算；销售统计页支持日期区间、分类和商品筛选，趋势图按天补齐无销售日期，避免折线断裂。

# 4.展示和测试

## 4.1 运行环境与启动方式

系统运行环境为 Node.js ≥ 20.19（Vite 8 要求，推荐 22 或 24 LTS）、MySQL ≥ 8.0、npm ≥ 10。首次运行在 backend 目录执行 npm install，复制 .env.example 为 .env 并填写数据库账号和 JWT_SECRET，执行 npm run db:init 完成建库、建表、种子数据和初始账号，再执行 npm run dev 启动后端；前端在 frontend 目录执行 npm install 和 npm run dev，访问 http://localhost:5173。初始化后默认账号为 admin、cangku01、shouyin01、caigou01、houqin01、kuaiji01，默认密码均为 123456，分别对应六类岗位。

为方便答辩演示，项目提供单端口模式和 Windows 离线演示包：先执行前端构建，再在 backend 的 .env 中设置 SERVE_FRONTEND=1，执行 npm start 后访问 http://localhost:3000 即可同时打开页面和接口；执行 node scripts/package-demo.js 可生成带便携 node.exe 的演示包，双击 start-demo.bat 自动创建 supermarket_demo 数据库，stop-demo.bat 停止服务，reset-demo.bat 重置演示数据。

演示视频保存在 docs/assets/demo.mp4（1920×1200，约 1 分 27 秒，8.6 MB），记录了登录、商品建档、收银开单、退货、库存盘点和采购付款的完整操作流程，可作为答辩现场演示的备份。

## 4.2 主要界面展示

（1）运营总览。如图 4.1 所示，页面顶部展示今日销售额、今日利润及环比，右侧卡片展示本月收货总数量、商品总种类、库存预警商品数和待补货商品数；中部是近 7 日销售金额趋势和商品分类库存占比；底部是库存余量 TOP5 和热销商品 TOP。看板数据由 /api/dashboard 聚合，利润字段仅管理员可见。

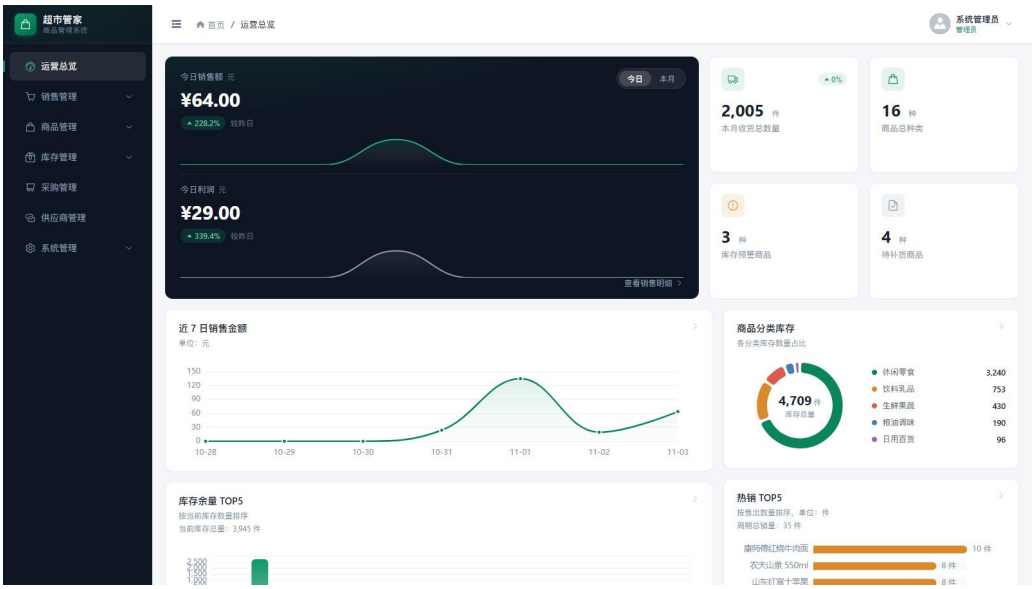



图 4.1 运营总览界面

（2）商品档案。如图 4.2 所示，列表展示商品名称、条码、分类、规格、加权成本、售价、库存、状态和供应商，库存低于预警阈值时显示“预警”标签；支持按名称、条码、分类和状态筛选，支持新增、编辑、删除、详情和导出档案。成本列对非管理员脱敏。

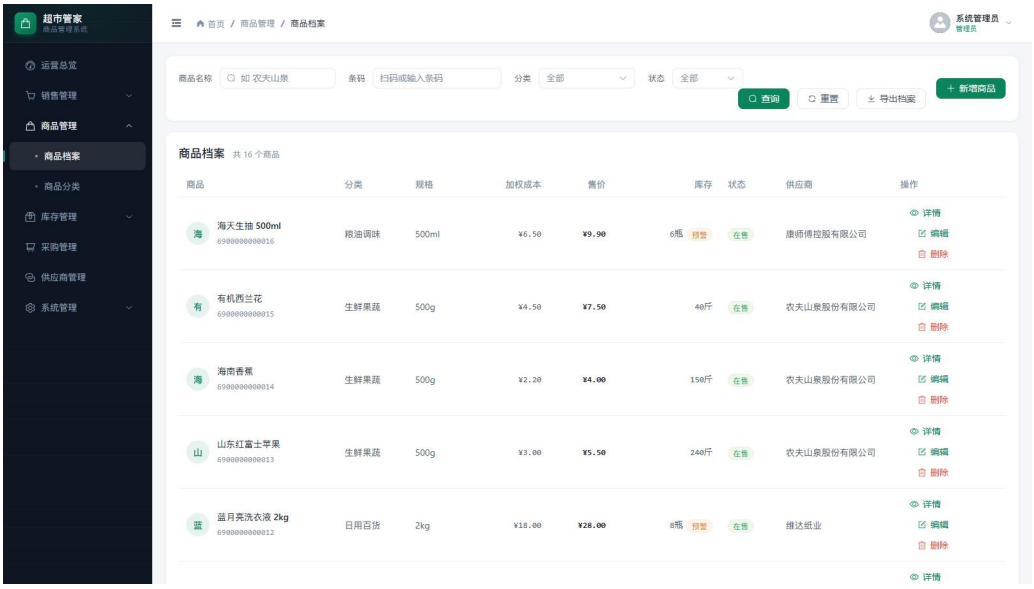



图 4.2 商品档案界面

（3）收银开单。如图 4.3 所示，左侧是商品卡片和搜索框，支持按名称/编码搜索和分类筛选，也可以使用扫码枪录入；右侧是购物清单，展示单价、数量、小计和应收合计；底部选择支付方式（微信、支付宝、现金、银行卡），输入实收金额后自动计算找零，点击“提交开单”后后端在事务内扣库存并生成销售单和出库流水。

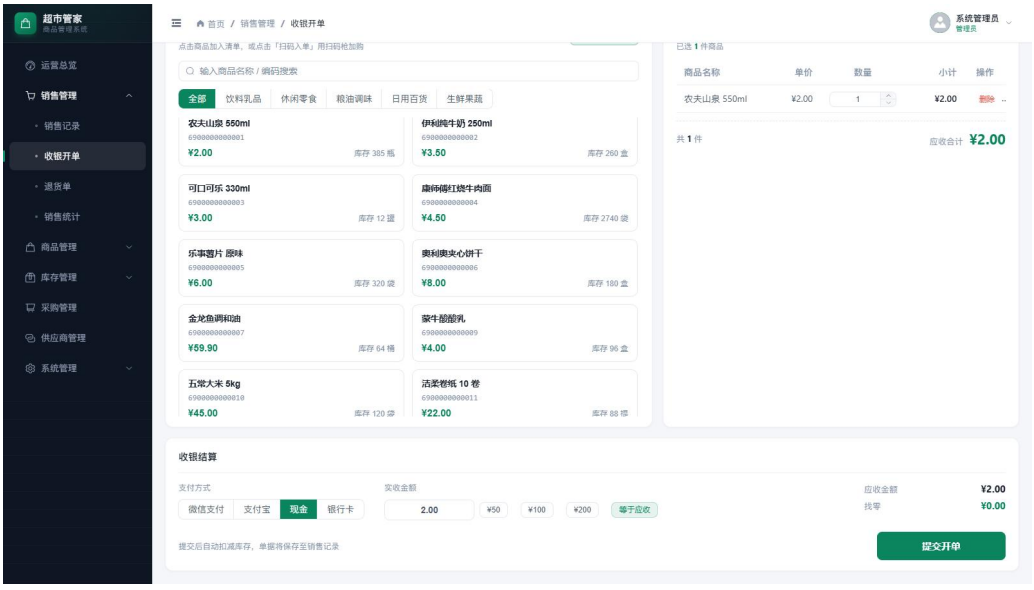



图 4.3 收银开单界面

（4）采购管理。如图 4.4 所示，列表展示采购单号、供应商、状态、采购金额、付款进度、制单人和创建时间；采购单按“草稿—已提交—已入库—已取消”流转，已入库单据显示应付和已付金额，管理员可登记付款。采购金额和付款数据只对管理员和采购岗位可见，仓管收货时只看到数量。

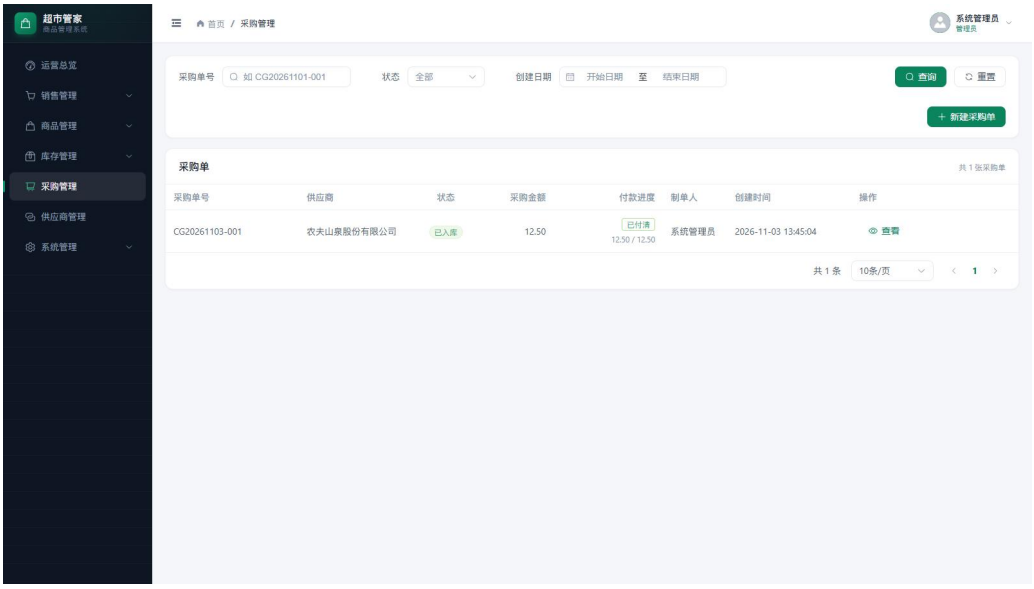



图 4.4 采购管理界面

（5）库存盘点。如图 4.5 所示，列表展示盘点单号、状态、盘盈/盘亏数量与金额、制单人和审核人；创建任务时按分类生成商品快照，草稿阶段录入实盘数和盈亏原因，提交后由管理员审核，审核完成自动修正库存并写盘盈/盘亏流水。非管理员查看盘点单时，成本和盈亏金额被脱敏，只保留数量。

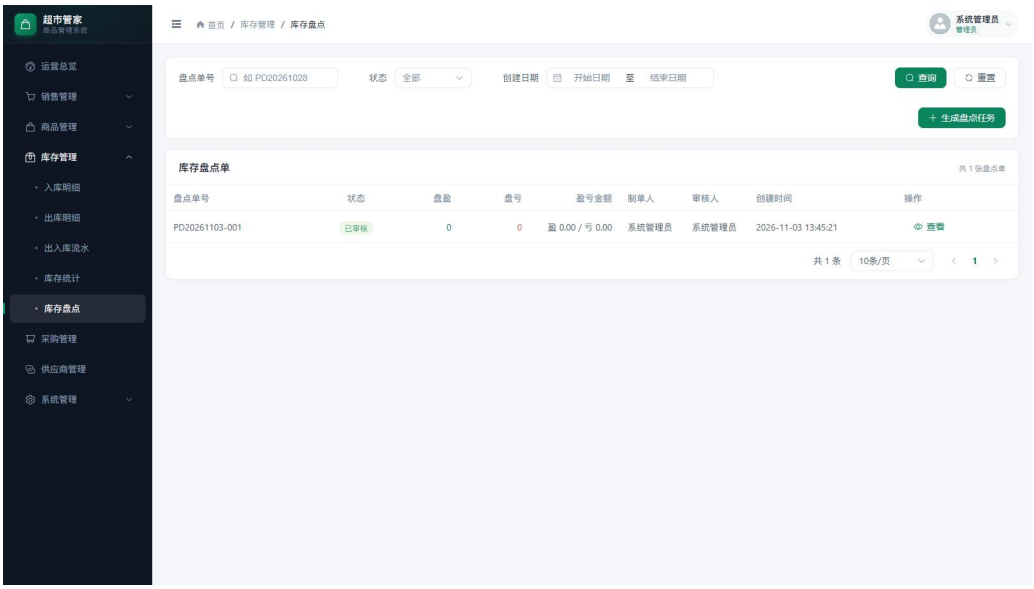



图 4.5 库存盘点界面

（6）操作日志。如图 4.6 所示，日志按时间倒序展示操作人、职位、模块、动作、业务单号、状态变化、操作内容和 IP；支持按模块、动作、操作人和时间筛选。销售开单、退货申请/确认、采购创建/提交/收货/付款、盘点创建/提交/审核、商品新增/修改等关键动作都会留痕，仅管理员可查看。




图 4.6 操作日志界面

## 4.3 测试方案与结果

### 4.3.1 后端单元测试

后端使用 Node.js 原生测试运行器，执行 cd backend && npm test，共 16 个测试文件、105 个用例，全部通过（105/105）。测试不依赖数据库，重点验证纯业务规则和边界条件，覆盖内容如表 4.1 所示。

表 4.1 后端单元测试覆盖

| 测试文件 | 覆盖内容 |
| --- | --- |
| weightedCost.test.js | 移动加权平均成本：首次入库、零库存、多次入库、未提供单价 |
| stockBiz.test.js | 库存流水业务类型与出入库方向 |
| stocktake.test.js | 盘点差异数量、差异金额、状态流转 |
| purchase.test.js | 采购金额计算、状态机、付款上限 |
| saleReturn.test.js | 退货状态、退货明细快照、重复确认 |
| saleItems.test.js | 销售明细合并、数量校验、金额取整 |
| auth / permissions.test.js | JWT 载荷、六类岗位权限矩阵 |
| financeVisibility.test.js | 成本、毛利、库存总值、采购金额脱敏 |
| userInvalidation.test.js | 改密、重置、禁用后旧 token 失效 |
| loginRateLimit / dateValidation | 登录限流阈值、日期区间格式与边界 |
| operationLog / productModel | 操作日志字段、成本不可手工修改 |
| configSecurity / stockWarn | 生产环境 JWT_SECRET 校验、库存预警判断 |

### 4.3.2 真实数据库冒烟测试

执行 cd backend && npm run test:smoke，测试会连接真实 MySQL，按“登录 → 看板 → 商品成本 → 入库与加权成本 → 销售扣库存 → 退货回补 → 采购收货 → 付款 → 统计 → 权限隔离”的顺序调用接口，验证数据库写入、事务提交和跨模块数据一致性。冒烟测试会写入业务数据，因此建议使用独立测试库，不要在生产库或演示库上执行。

### 4.3.3 浏览器端到端测试

参考 Node.js 在线测试系统的组织方式[10]，执行 cd frontend && npm run test:e2e，脚本自动创建 supermarket_e2e_<时间戳> 临时数据库，启动独立后端（3210）和前端预览（4173），使用本机 Chrome 或 Edge 连续完成 12 步操作，结束后删除临时库，不污染演示数据。12 步流程如表 4.2 所示，最终 12/12 通过且无控制台错误，失败截图保存在 frontend/test-results。

表 4.2 浏览器 E2E 流程

| 步骤 | 验证内容 | 关键断言 |
|---|---|---|
| 1 | 管理员登录 | 登录成功并进入运营总览 |
| 2 | 新增商品 + 期初入库 + 库存校验 | 商品可查、库存与流水一致 |
| 3 | 入库/出库切换 + 列表重载 | 两种流水方向正确展示 |
| 4 | 销售开单 + 退货申请/确认 | 库存扣减、退货回补、原单状态变化 |
| 5 | 库存盘点：生成/保存/提交/审核 | 盈亏原因必填、审核后库存修正 |
| 6 | 采购：制单/提交/收货/付款 | 状态流转、应付与已付金额正确 |
| 7 | 操作日志记录盘点审核 | 模块、动作、单号、状态变化可查 |
| 8 | 积压商品 + 供应商进货排行 | 统计结果与业务数据一致 |
| 9 | 收银员权限与本人销售单 | 只能看本人单据、不能看成本 |
| 10 | 仓管菜单与销售权限拦截 | 无销售菜单、接口返回 403 |
| 11 | 会计菜单与成本报表隔离 | 可看全部销售单、成本字段脱敏 |
| 12 | 浏览器无控制台错误 | 全程无 error 级日志 |

### 4.3.4 持续集成与结果汇总

项目配置 GitHub Actions，在每次 push 或 pull request 到 main 时执行：安装 Node.js 24 和后端依赖、运行 105 个单元测试、启动 MySQL 8.0 服务并执行 db:init、运行真实接口冒烟测试。三类测试的结果汇总如表 4.3 所示。

表 4.3 测试结果汇总

| 验证层级 | 执行方式 | 结果 |
|---|---|---|
| 后端单元测试 | cd backend && npm test | 105/105 通过 |
| 真实数据库冒烟 | cd backend && npm run test:smoke | 登录、库存、销售、退货、采购、付款、权限链路通过 |
| 浏览器 E2E | cd frontend && npm run test:e2e | 12/12 通过，无控制台错误 |
| 持续集成 | GitHub Actions | 单元测试 + MySQL 初始化 + 冒烟测试通过 |

## 4.4 测试中发现并解决的问题

在联调和测试过程中，以下问题被重点处理。

（1）并发超卖。最初逐行校验库存，同一商品多行时合计可能超过库存；改为先合并明细再校验，并在事务内对商品行加 FOR UPDATE 锁，按商品 id 升序加锁。

（2）单号重复。并发结账时“日期 + 序号”可能生成相同单号；为单号字段加唯一索引，捕获重复键后有限次重试，避免把数据库异常直接抛给用户。

（3）旧 token 仍可访问。最初只校验 JWT 签名和过期时间，改密后旧 token 仍有效；在用户表增加 token_version，签发时写入载荷，每次请求回库比对，改密、重置、改岗位或禁用后自增，旧 token 立即失效。

（4）成本被手工改。商品编辑页曾允许提交 cost_price，导致加权成本失真；后端增加“成本只能由入库自动计算”的校验，前端提交原值可以通过，提交修改值直接报错。

（5）销售统计按分类筛选失效。销售明细只保存分类名称快照，没有分类 id；统计接口先把 categoryId 换算成分类名，再按名称过滤，查不到分类时返回空结果而不是全量数据。

（6）跨时区“今天”偏移。服务器时区与 MySQL 会话时区不一致时，今日销售额可能统计到前一天；统一设置 TZ=Asia/Shanghai 和 DB_TIME_ZONE=+08:00，并在日期工具中统一计算日边界。

（7）退货与作废重复回补。退货确认和销售作废都会回补库存，如果并发执行可能回补两次；两者都先锁原销售单，作废时取消该单下待确认的退货申请，保证只有一条路径生效。

# 5.创新点与不足

## 5.1 主要特点

（1）单据驱动库存，库存全程可追溯。系统不提供“直接改库存”的入口，所有库存变化都必须来自入库、销售、退货、采购收货、盘点或作废单据，并生成对应流水。任意时刻的库存都可以由流水解释，盘点差异也能定位到具体业务类型。

（2）事务、行锁和锁顺序共同保证一致性。销售、退货、采购收货和盘点审核都在事务内完成“单据 + 库存 + 流水”的更新；涉及多商品时按商品 id 升序加锁，降低死锁概率；单号冲突用唯一索引和有限重试兜底。

（3）成本快照与移动加权平均结合。入库自动更新加权成本，销售时保存成本快照，退货复制原成本快照，盘点按审核时成本计算差异。这样毛利可复盘，历史单据不会因后续采购价变化而被改写。

（4）权限控制落到后端和字段级。系统不仅有菜单和路由控制，还在接口层校验权限码；成本、毛利、库存总值、盘点盈亏金额、采购金额按岗位脱敏，非管理员直接调用接口也拿不到敏感字段；tokenVersion 让改密、禁用和改岗位后的旧 token 立即失效。

（5）质量保障可验证。105 个单元测试覆盖成本、库存、退货、采购、盘点、权限、脱敏、限流和日期边界；真实 MySQL 冒烟测试验证事务和跨模块链路；12 步浏览器 E2E 验证真实点击流程；GitHub Actions 在每次提交后自动回归。相比只提供页面截图的课程作业，这套验证链路更能说明系统“真的能跑通”。

（6）低成本交付。系统支持开发态前后端分离、演示态单端口托管和 Windows 离线演示包三种运行方式；演示包自带便携 node.exe，不要求演示机安装 Node，只需本机已启动 MySQL，适合答辩现场使用。

## 5.2 不足与改进方向

（1）只支持整单退货。当前退货单按原销售单整单退回，不支持按商品部分退货，也不支持多次退货。改进方向是把退货明细与销售明细建立数量关联，增加“已退数量”和“可退数量”，并在确认时按明细回补库存、按比例冲减营收。

（2）采购付款没有独立流水。应付金额和累计已付挂在采购单上，无法按付款时间、付款方式、发票号对账。改进方向是增加 payment_record 表，记录每次付款的金额、方式、操作人和凭证，采购单只保留汇总金额。

（3）单店单仓，缺少批次和保质期。当前一个商品只有一个库存数量，不能区分不同进货批次和保质期，也无法支持多门店调拨。改进方向是增加 warehouse、stock_batch 表，把库存从“商品维度”细化为“商品 + 仓库 + 批次”维度。

（4）没有物理外键。表间关系由应用层校验，数据库层面无法完全阻止脏数据。改进方向是在核心表上补充外键和级联策略，或在应用层增加统一的数据校验入口。

（5）测试覆盖不均衡。后端单元测试较完整，但前端以 E2E 为主，缺少组件级单元测试；图表和大数据量列表没有做压力测试。改进方向是引入 Vitest 和 Vue Test Utils 补充组件测试，使用压测工具验证分页和统计接口在万级数据下的响应。

（6）部署形态有限。离线演示包仅支持 Windows x64，未提供 Docker、云数据库和对象存储方案；JWT 黑名单保存在单机内存，多实例部署需要替换为 Redis。改进方向是补充 Dockerfile 和 docker-compose，提供一键部署脚本，并把黑名单和限流迁移到 Redis。

（7）业务范围有限。系统不包含会员、优惠券、线上支付、电商订单、批次/序列号管理等商业系统能力，定位是单店进销存后台，而不是完整的零售中台。这一点在需求阶段已经明确，后续可根据实际经营需要逐步扩展。

# 6.项目总结

本项目从需求调研、数据库设计、接口开发、前端实现到测试和交付，完整走了一遍小型管理系统的工程流程。项目最终实现 14 张业务表、69 个接口、8 个功能模块和 6 类岗位权限，覆盖商品、供应商、库存、采购、销售、退货、盘点和经营分析，能够跑通“商品建档 → 期初入库 → 收银开单 → 退货 → 盘点 → 采购收货 → 付款”的完整闭环。

在开发过程中，最大的困难不是页面数量，而是数据一致性。最初按“页面需要什么就写什么接口”的方式开发，出现了销售扣库存与写流水不同步、退货和作废重复回补、并发结账超卖等问题。后来把库存变化统一收敛到单据和流水，把事务边界放在 Controller，把跨模块逻辑抽到 Service，并固定商品加锁顺序，问题才逐步收敛。这个过程让我理解了“业务规则写在后端、前端只做展示和轻量校验”的必要性。

第二个困难是权限边界。系统有六类岗位，菜单、路由、接口和字段四个层面都要控制。最初只在前端隐藏按钮，测试时发现直接调用接口仍能拿到成本数据；后来把权限码下沉到后端路由，并增加字段级脱敏和 tokenVersion，才做到“看不到的也拿不到”。这让我意识到，安全设计不能只依赖界面。

第三个困难是测试环境。E2E 如果直接使用开发库，会污染演示数据；如果只做接口测试，又无法验证真实点击流程。最终采用“临时数据库 + 独立后端端口 + 前端预览 + 本机浏览器”的方案，测试结束自动删库，既保证真实性，又不影响演示数据。GitHub Actions 的接入则让每次提交都能自动回归，减少了“改一处、坏一处”的情况。

通过这个项目，我在数据库设计、事务与并发、权限控制、前后端接口契约、测试与持续集成等方面都有较大提升，也学会了用 README、接口文档、设计规范和演示包把项目交付给他人。后续如果继续迭代，我会优先实现部分退货、采购付款流水、批次与保质期管理，并补充 Docker 部署和前端组件测试，让系统从“能跑通的课程设计”逐步接近“可长期使用的小型门店后台”。

# 7.参考文献


[1] 袁明坤,曾丽.基于 JavaWeb 的超市商品采购管理系统设计与实现[J].电脑知识与技术,2022,18(5):61-63.

[2] 杨正午.基于 WEB 前端开发技术的网站设计——以连锁超市商品销售管理系统为例[J].山西科技,2019,34(3):51-53+57.

[3] 赵霁雯.中小型超市库存管理系统设计及实现[D].大连:大连理工大学,2018.

[4] 崔可飞.超市商品管理系统的设计与实现[D].长春:吉林大学,2016.

[5] 茅希慧.基于 B/S 架构的超市商品信息管理系统的研究与设计[D].成都:电子科技大学,2014.

[6] 阳武.基于 Node.js 和 Vue 的房屋租赁管理系统研究与实现[J].山西电子技术,2026,4(4):70-73.

[7] 项露莎,孙佳怡,李梦婷.基于 Vue 和 Node.js 的音乐门票管理系统的设计与实现[J].现代信息科技,2025,9(11):96-101.

[8] 黄苗苗.基于 Vue 和 Node.js 的作业管理系统的设计与实现[J].现代信息科技,2024,8(22):102-105+110.

[9] 黎青霞.Node.js 在 Web 开发中的应用研究[J].信息记录材料,2024,25(10):91-93+96.

[10] 李淑珍,朱彤.基于 Node.js 技术的在线测试系统设计方案[J].科技资讯,2023,21(19):35-38.

[11] 霍春阳.Vue.js 设计与实现[M].北京:人民邮电出版社,2022.

[12] 王珊,萨师煊.数据库系统概论[M].第 5 版.北京:高等教育出版社,2014.

[13] JONES M, BRADLEY J, SAKIMURA N. JSON Web Token (JWT): RFC 7519[S]. Internet Engineering Task Force, 2015.

# 8.致谢

行文至此，毕业实践综合项目即将收尾。回顾从选题、开题到系统实现和报告撰写的整个过程，我得到了许多人的帮助，在此表示衷心的感谢。

首先感谢我的指导教师李杰老师。从选题方向的确定、开题报告的修改，到系统功能取舍、数据库设计和报告结构，李老师都给出了耐心而具体的建议。每当我陷入“功能越多越好”的思路时，李老师都会提醒我先保证核心业务闭环和代码质量；在报告修改阶段，李老师也逐段指出表述不清和格式不规范的地方，使这篇报告得以成形。

感谢信息与智能工程学院的各位任课教师。正是他们在 Web 开发、数据库、软件工程等课程中的讲授，让我具备了完成这个项目所需的基础知识；也感谢辅导员老师在学习和生活上的关心与提醒。

感谢一起学习和讨论的同学。在系统测试阶段，几位同学帮助我试用收银、退货和盘点流程，提出了不少从使用者角度出发的改进意见；在遇到 Express 中间件、MySQL 锁和 Vue 路由问题时，和同学的讨论也让我少走了很多弯路。

感谢我的家人。他们在我求学期间一直给予我理解和支持，让我可以安心完成学业和毕业设计。

最后，感谢所有参与评审和答辩的老师。由于个人水平有限，系统中还存在整单退货、付款流水、批次管理等方面的不足，恳请各位老师批评指正。我会把这次毕业实践作为新的起点，在今后的学习和工作中继续完善自己。